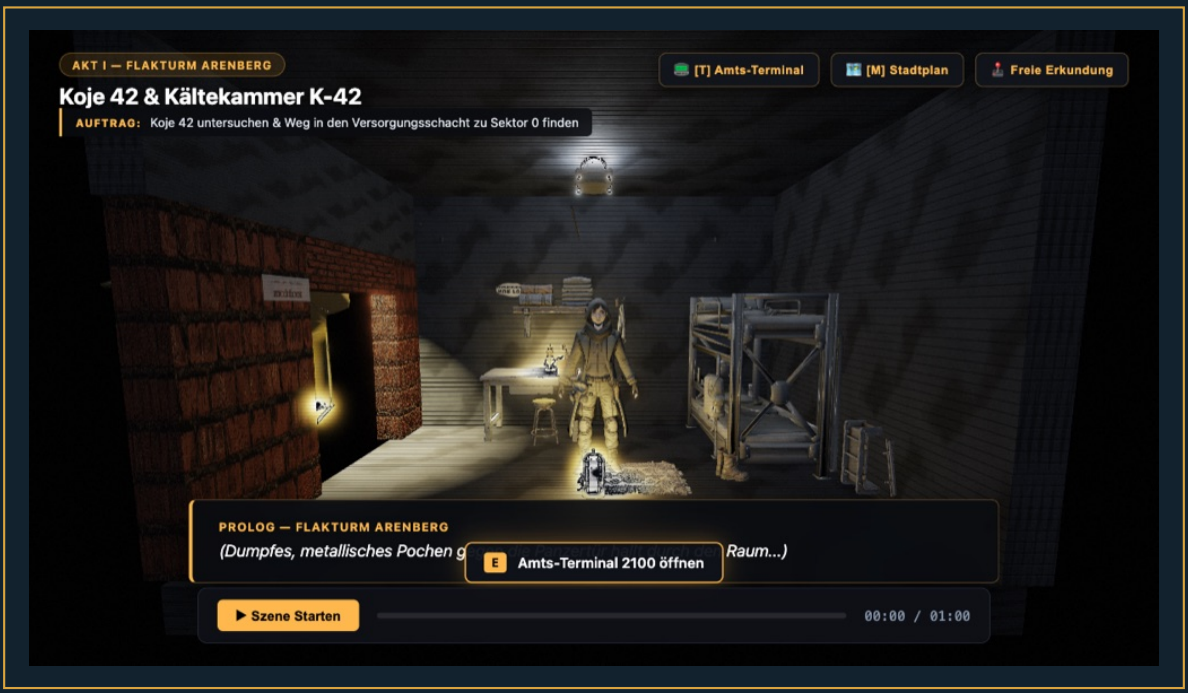


Small World Machine

Eine leichte 3D-Engine für das offene Web – Dossier

Konzept, Sinn, Fähigkeiten und ein ehrlicher Technikvergleich

Release v0.111.0 · Stand 17.10.2026

Eckdaten

Status:	aktive Entwicklung, nicht publiziert (privates npm-Workspace)
Lizenz:	MIT
Technik:	strikt typisiertes TypeScript, hybride WebGPU / WebGL 2 / WebGL 1-Engine
Autor:	Stefan Rottensteiner (kleines Indie-/Solo-Projekt)

1 Was ist Small World Machine?

Small World Machine ist eine extrem leichtgewichtige, 3D-Engine für den Browser, geschrieben in strikt typisiertem TypeScript. Sie besteht aus einem einzigen, von Dritten abhängigkeitsfreien Kern („Zero Dependency“-Fördergrundsatz – kein three.js, kein Babylon.js) und deckt gleichzeitig drei Grafik-Backends ab: WebGPU, WebGL 2 und WebGL 1. Das Projekt versteht sich als „Preact der 3D-Engines“: architektonisch sauber, klein, und bewusst ohne den Ballast großer Frameworks.

Systemkomponenten wie Behavior-System, FSM, InteractionManager, AudioSystem, Impuls-Physik sowie die Asset- und Level-Pipeline sind keine nachgerüsteten Plug-ins, sondern Teil des Kerns. Damit grenzt sich das Projekt bewusst von reinen Rendering-Bibliotheken ab – siehe der detaillierte Vergleich in Abschnitt 5.

2 Sinn & Zweck: Wozu die Engine existiert

Das Leitbild des Projekts ist im Dokument „The Small World Machine Vision“ formuliert: Atmosphäre, Kunstrichtung und Game-Feel schlagen rohe Hardwarespezifikationen. Angelehnt an Titel wie Stray, Inside oder Journey will die Engine kleine, dichte, atmosphärische Erlebnisse ermöglichen – ohne 2.000-Dollar-GPU und ohne 50-GB-Download.

Monolithische Engine-Tax hinter sich zu lassen. Drei Schmerzpunkte adressiert die Engine ausdrücklich:

- **Monolithische Engine-Steuer abwerfen:** Keine Megabytes an un-treeshakablem Runtime-Overhead, keine Multi-Gigabyte-Installationen – ein kompletter Engine-Stack in Razor-Thin-Footprint für das offene Web.
- **Echte Spielarchitektur statt „nur ein Renderer“:** Game-Loop, Zustandsmaschinen, Physik, räumliches Audio und Pointer-Interaktion sind eingebaut – Teams müssen sich nichts zusammenkleben.
- **Hybrid ohne Lock-in:** WebGPU-Compute-Pipelines (z. B. Clustered Forward+ Light Culling) wo verfügbar, mit sauberen Abwärtskaskaden auf WebGL 2 / WebGL 1.

Dazu kommt ein dezidiertes Entwicklerdienstrünken: 100 % striktes TypeScript, keine impliziten „any“, Fail-Fast-Lebenszyklen, Objekt-Pooling und Zero-Allocation-Hot-Paths sowie ein automatisches GPU-Ressourcenmanagement per Referenzzählung – kein manuelles dispose()-Gefummel mehr.

3 Fähigkeiten (Auszug)

3.1 Hybrid-Rendering-Pipeline

- Physikalisch basiertes Rendering (Cook-Torrance-BRDF), lineares Farbschema mit sRGB-Gamma-Korrektur über den gesamten Pfad.
- Beleuchtung: Ambient, Directional, Point, Spot, Area; Clustered Forward+, Clustered/Tiled Per-Cell-Light-Listen – auf WebGPU per Compute, auf WebGL 2 per CPU.
- Schatten: Cascaded Shadow Maps, Hardware-Shadow-Sampling, weiche PCSS-Schatten (Directional), Schatten-Kaskaden mit Texel-Snapping.
- Reflexionen: planare Boden-Reflexion und dynamische sphärische Inversions-Reflexionen; IBL (Irradiance/Radiance) mit eigenen Generatoren.
- Volumetrischer Froxel-Nebel (Stufe 3, v0.109–v0.111): 3D-Froxel-Raster mit Raymarch-Z-Integration, FogVolume-Interior-Carving, Dichte-Masken, 3D-Noise (WebGPU-Compute; WebGL 2 via Halb-Auflösungs-Raymarch + bilateralem Upsampling).
- Volumetrische Lichtsäulen / Light Shafts („God Rays“) als hybrider Screen-Space-/Shadow-Map-Pass über alle drei Backends.

Moderner WebGPU-Pfad wird dabei nie abgespeckt: Compute-Pipelines, 3D-Froxel-Grid und Temporal-Reprojection bleiben vollwertig; WebGL 2 und WebGL 1 erhalten bestmögliche eigene Fallbacks statt einer geteilten Minimalstufe.

3.2 Materialien & Post-Processing

- Standard-PBR (Metallic/Roughness), Glas/Dielektrika mit Screen-Space-Refraktion, IOR und Beer’s-Law-Absorption; Flüssigkeiten (OpenWater/StylizedWater, Lava, Slime).

• Stilisierte Looks: ACES-Filmic/Reinhard/Cineon-Tonemapping, parametrisches Farbgrading, Bloom, HBAO, vereinfachtes TAA, Motion-Trails sowie Retro-Style-Effekte (Noir, VHS, Thermal, Phosphor, Cyber-Glitch).

• Hochpräzises Box-Decal-System über einen dünnen Normal-Buffer (WebGPU/WebGL 2).

3.3 Systeme für Gameplay & Interaktion

• Behavior-System (addBehavior) statt Controller-Arrays – Controller, Hover-, Draggable-, Proximity-, Stage-Bewegungen usw. sind Behaviors.

• Typsichere, zero-allocation State-Machines (FSM) inkl. StateMachineBehavior.

• InteractionManager mit O(log n)-Octree-Picking und Möller-Trumbore-Raycasts.

• Impuls-Physik: SAT-Kollisionen, Auftrieb, Sub-Stepping, Joint-System (Distance/Spring/BallSocket/Hinge).

• AudioSystem: 3D-räumliches Audio (HRTF), prozeduraler Retro-Synthesizer (Füsse, Laser, Drones, Feuer), Mixer mit prozeduralem Reverb.

4 Architektur & Qualitätssicherung

Die technischen Fundament-Entscheidungen sind dokumentiert (Architecture Decision Records, ADRs) und werden konsequent durchgesetzt:

• **Keine globalen Singletons:** Die Engine unterstützt mehrere Instanzen pro Seite; Abhängigkeiten laufen über Context-Objekte, Konstruktor-Injektion und Lifecycle-Methoden statt über statischen Zustand.

• **Strikte Typen, kein „any“:** Explizite Zugriffsmodifier und Typen; der Linter blockiert implizite Typ-Lücken sofort.

• **Fail-Fast mit Augenmaß:** Ungültige Zustände werden dort gemeldet, wo ein Subsystem sie tatsächlich verarbeitet – nicht vorzeitig in Settern.

• **Ressourcen-Sicherheit:** Automatische GPU-Ressourcenfreigabe per Referenzzählung, Objekt-Pooling (MathPool), Zero-Allocation-Hot-Paths, engine.destroy() zur vollständigen Freigabe.

• **Determinismus:** Ein Capture-Modus (captureFrameRate) schreitet den Loop in exakten Festschritten voran – reproduzierbare Tests, Screenshots und Aufnahmen auf derselben Umgebung.

• **Basis Universal / KTX2:** WASM-basiertes Transcoding für GPU-native Formate (BC7, ASTC, ETC2, BC3) statt ballastiger Textur-Konvertierung.

4.1 Verifikation & Testbasis

Das Projekt wird über eine umfangreiche automatisierte Qualitätsbasis abgesichert: eine große Testsuite (typischerweise >1.500–1.900 Tests, Stand der Release-Zyklen) über Math, Physik, Shader-Invarianten und Color/Pipeline-Korrektheit; Backbone-Showcases, die auf allen drei Renderern gegen Goldener-Bilder bzw. numerische Assertions geprüft werden; eigene Lint-Pipelines für WGSL/GLSL sowie deterministische Capture-Tests für Post-Processing-Pfade. Reproduzierbarkeits-Läufe laufen gegen eingefrorene Builds, nicht gegen den Live-Editor-Baum.

4.2 Beispielprojekte & Showcases

• »The Whisper – A Viennese Requiem«: atmosphärisches Prolog-Erlebnis mit deklarativen Level-Deskriptoren, Asset-Kits und Licht-Staging.

• »Light Cycle Arena« & »YAD«: Sample-Apps, die Spielmodi auf der Engine demonstrieren.

• Über 39 nummerierte Showcases (1–38 + 39 „Sanctum“) decken PBR, IBL, Glas-Transmission, Post-Processing, Clustered-Lighting, räumliches Audio, volumetrischen Nebel, Decals und mehr ab.

5 Einsatzszenarien & Entwicklungs-Ökosystem

Das Projekt liefert nicht nur den Kern, sondern eine komplette, browserbasierte Werkzeugkette sowie deklarative Inhaltsformate:

• „Maker“: eigenständiger 3D-Welt-Editor im Browser (File System Access API), glTF-2.0-Persistenz mit SWM_*-Erweiterungen, Undo, Grid-Snapping, Prefab-Pipeline, Content-Browser sowie ASCII-Karten-Import für Dungeon-Level.

- The Forge: dockbares In-Game-Overlay für Runtime-Inspektion und Diagnostik.
- Werkzeuge: Material-Studio (PBR-Maps), Pixler (Pixel-Editor), Xtractor (Sprite-Slicing), Map-Generator (ASCII-Grid), IBL-Generator, Gamepad-Diagnose.
- Deklarative Level-Deskriptoren (level.schema.json) + KitRegistry mit modularen Asset-Kits, Socket-Licht-, Decal- und Verhaltens-Bindings.
- Asset-Pipeline: glTF 2.0 (mit Erweiterungen), Basis Universal/KTX2-Transcoding per WASM, OBJ/MTL; Einbindung CC0/CC-BY-Assets (Kenney, Poly Haven, Tripo3D).

Diese Offenheit führt zu einem klaren Zielprofil: atmosphärische Indie-Spiele und interaktive Erlebnisse für den Browser – Playable Ads, Web-Demos, Kurztitel, interaktive 3D-Inhalte auf Webseiten. Eine kommerzielle Roadmap beschreibt zudem den Weg über Tauri v2 für Steam-Veröffentlichungen sowie Portierungspartnerschaften für Konsolen.

6 Ehrlicher technischer Vergleich mit etablierten Systemen

Kriterium	Small World Machine	Godot 4	three.js	Unreal Engine 5
Typ	3D-Engine (Kern) für den Browser	Vollwertige Game-Engine + Editor	3D-Ausgabekontext (Bibliothek)	AAA-Game-Engine + Editor
Web-Auslieferung	nativ im Browser (WebGPU / WebGL 2 / WebGL 1)	WebAssembly + WebGL 2 (Compatibility; kein WebGPU im Web-Export)	nativ im Browser (WebGL; WebGPU experimentell)	kein nativer Browser-Export; üblicherweise Pixel Streaming (Cloud-Rendering)
Game-Loop / Engine-Services	im Kern integriert (Loop, FSM, Physik, Audio, Interaktion)	integriert (Scenes, Nodes, eigene Physik/Audio)	nicht enthalten – wird selbst gebaut oder über Add-ons ergänzt	integriert, sehr umfangreich
Sprache(n)	TypeScript (strikt)	GDScript, C#, C++	JavaScript / TypeScript	Blueprint + C++
Footprint / Komplexität	sehr klein, keine Kern-Abhängigkeiten	kompletter Editor (Gigabyte-Installation), Webfat-Paket groß	Bibliothek klein, Ökosystem riesig	sehr groß; eigener Editor, mächtig, komplex
Community & Assets	klein, projekteigene Kits/CC0-Basis	groß, Asset-Library, aktive Community	sehr groß, ext. Beispiel-Basis	sehr groß, Marktplatz
Lizenz	MIT, privat (noch nicht veröffentlicht)	MIT (open source)	MIT	Kommerziell (Royalty ab Umsatzschwelle)
Zielprofil	kleine atmosphärische Web-/Indie-Titel, Playable Ads, Lade-Effizienz	Indie-/Mittelgroße Titel, mobil & Desktop, Web als Nebenschauplatz	Web-3D-Inhalte, Tooling, Prototypen, Szenen ohne Engine-Anspruch	große, hochglanzige AAA-Projekte native Plattformen

Quelle für die Spalte „Godot 4“: offizielle Godot-Dokumentation „Exporting for the Web“ (Stand 2026): Web-Export nur via WebAssembly + WebGL 2 (Compatibility-Renderer), kein WebGPU im Web-Export, Standard ist Single-Threaded-Export, C#/GDExtensions nicht auf allen Plattformen im Web verfügbar, Audio-Effekte/Reverb im Web eingeschränkt. „WebGPU“ bleibt dort Desktop/Mobile vorbehalten. three.js stellt WebGPU als experimentelles Add-on bereit; ein vollwertiger Engine-Stack (Loop, Physik, Audio, Editor) ist nicht Bestandteil. Unreal für den Browser bedeutet in der Praxis Pixel Streaming, nicht nativ gerendertes HTML5/WASM.

6.1 Ehrliche Einschätzung der eigenen Grenzen

Small World Machine ist bewusst schmal aufgestellt und bildet keine AAA-Bücher nach:

- **Kein volles LOD**-System, **kein volles TAA** (vereinfachte Variante ohne Motion-Vector-Reprojektion), **kein GTAO** (nur vereinfachtes HBAO).
- WebGL 1 ist Legacy-Fallback ohne Schatten/Volumetrik; Feature-Parität ist nicht „bit-exakt“ über Geräte hinweg garantiert (deterministisch nur bei identischer Umgebung).
- Kein Multiplayer-/Backend-Framework im Kern, keine visuelle Blueprint-ähnliche Node-Programmierung, kein eigener Blender-artiger Animations-/Charakter-Editor – Rigging erfolgt extern per glTF/Mixamo-Pipeline.
- Einzelentwickler-Projekt (v0.111), noch nicht publiziert, kleine Community und kein großer Asset-Marktplatz; das Ökosystem wächst primär über CC0/CC-BY-Kits.
- Qualitätsstufen für schwache Mobilgeräte (automatisches Downscaling / Post-Processing-Reduktion) sind ausdrücklich noch offen (siehe Backlog).

6.2 Wann welches System sinnvoll ist

• **Small World Machine:** wenn das Ziel „kleiner, dichter, atmosphärischer Web-Inhalt“ lautet – Playable Ads, Web-Demos, Kurztitel, interaktive Szenen; wenn Footprint und Lade-Timing dominieren und modernes WebGPU dennoch voll genutzt werden soll.

• **Godot 4:** wenn ein vollwertiges, eigenständiges Spiel entstehen soll und Desktop/Mobile im Vordergrund stehen; für Browser-Auslieferung ist der Compatibility-Pfad funktional, aber weniger leistungsfähig.

• **three.js:** wenn maximaler Gestaltungsspielraum als Bibliothek gefragt ist und man Gameplay-, Physik- und Audio-Schicht selbst zusammenstellt; ideal für Web-Tooling, Produkt-3D und Prototypen.

• **Unreal 5:** wenn ein AAA-ambitioniertes, native Zielplattformen anvisiert werden und Hardware-Ressourcen sowie Ladezeiten keine Rolle spielen; Browser ist außerhalb von Pixel Streaming praktisch kein Fokus.

6.3 Fazit

Small World Machine beansprucht keine Alleinherrschaft – es beantwortet eine spezifische Frage: Wie baut man kleine, atmosphärisch dichte 3D-Erlebnisse mit maximaler Lade-Effizienz und moderner WebGPU-Grafik im offenen Web, ohne Engine-Monolithen und ohne Kompositions-Chaos? Wo diese Frage im Raum steht, liefert das Projekt einen – für seine Größe bemerkenswert – vollständigen Werkzeug- und Systembestand. Wo hingegen vollwertige Autoren-Umgebungen, große Teams und native AAA-Pipelines Verpflichtung sind, bleiben Godot, three.js und Unreal die angemessenen Werkzeuge.

Dieses Dossier basiert auf Projektdokumentation, ADRs, Guides, Research-Papieren sowie eigenem Recherche-Stand (u. a. offizielle Godot-Web-Export-Dokumentation, Stand 2026).

Alle weiterführenden Punkte sind im Repository unter docs/ sowie in CONTEXT.md, VISION.md und den ADRs hinterlegt.